

Beckett Lee-Messer

San Jose, CA | (408) 784-1694 | beckett.lee-messer.ug@dartmouth.edu
[linkedin.com/in/beckett-lee-messer](https://www.linkedin.com/in/beckett-lee-messer) | github.com/bleemesser

EDUCATION

Vassar–Dartmouth Dual Degree Program (2-1-1-1): Physics BA at Vassar, Computer Engineering BE at Dartmouth.

Dartmouth College – Hanover, NH

Expected graduation: June 2029

Area of study: Computer Engineering (BE) | GPA: 4.0

In residence from summer term 2026 (sophomore summer)

Coursework: Systems, Intro to Engineering; *in progress:* Software Design & Implementation, Linear & Digital Circuits

Vassar College – Poughkeepsie, NY

Expected graduation: May 2028

Area of study: Physics (BA) | GPA: 3.98 | STEM: 4.0

Coursework: Theory of Computation, Data Structures & Algorithms, Object-Oriented Programming, Functional Programming, Linear Algebra, Differential Equations, Electricity & Magnetism

TECHNICAL SKILLS

Languages: Rust, C++, C, Python, Go, Java, OCaml

Embedded: Bare-metal firmware, SPI, UART, GPIO, interrupts/ISRs, memory-mapped I/O, mmWave radar, emulation

Tools: Git, Docker, Linux CLI, AWS/GCP, CI/CD, Autodesk Fusion (CAD)

PROJECTS

IWRL6432 Bare-Metal Radar Firmware

Jun 2026 – Present

- Wrote firmware for a Texas Instruments mmWave radar SoC in pure Rust, with no vendor C code linked in. Reconstructed the RF front-end bringup and mailbox sequences from SDK sources and a JTAG register diff against a running reference binary.
- Built the HAL from scratch (clock and APLL, pinmux, GPIO, SPI, UART, ADC capture). Streamed raw ADC samples over a 20 MHz SPI link at 1.4 MB/s and 600 chirps/s, using on-device bit-packing and random subsampling with L1 reconstruction on the host, and a serial CLI to switch profiles and control capture.

mmWave Incontinence Detection – Stanford Medicine (Dr. David Svec)

Sep 2026 – Present

- Building the Python detection stack for a bedside radar sensor meant to flag patient incontinence in real time, in place of slow nursing rounds: slow-time SVD clutter removal, M-SBL sparse angle estimation per range bin, a three-state occupancy HMM, and a minimum-expected-cost decision over dry, wet object, and wet person.

Notesofast

Jun 2026 – Present

- Built a GPU-accelerated note-taking app for iOS in Rust and WGSL with a Swift shell: custom handwriting renderer, tiled PDF rendering, PDF import and export, encrypted storage, and backup to WebDAV or any S3-compatible bucket. In TestFlight, with an App Store build approved.

Edfarray

Mar – Sep 2026

- Built a Rust library with Python bindings that reads EDF, EDF+C, and EDF+D files, the formats that store EEG recordings, and published it to PyPI with docs. Exposes numpy-style indexing and 2D/3D proxies over same-rate channel groups, so ML code batches records without loading the whole file.
- Reads single signals 14-93x faster than pyedflib depending on access pattern, and pages a 10-second, 64-channel window 47x faster (warm cache). Chooses between a memory map and a bounded sequential read by how much of the range is resident, keeping an 8.5x lead on cold reads from a 4 GiB file.

6502 CPU Emulator

Jan – Mar 2025

- Built a cycle-accurate emulator for the 8-bit 6502 processor from the original datasheet and verified it against a standard accuracy test suite. It runs a lightly modified WozMon for the Apple I, and it can map arbitrary hardware into memory.

WORK EXPERIENCE

Machine Learning & Software Engineering Intern – Nuha

Jan 2025 – Feb 2026

- Built a Python library over a C++ backend that configures Texas Instruments radar sensors and streams their data in real time, then the pipeline on top of it that feeds per-person position tracking to an LLM, so the product can talk with several people at once.
- Rewrote the client from Python to Rust, covering speech detection and recording, sensor input, and audio playback. Fixed timing and concurrency bugs, replaced ad-hoc control flow with an explicit state machine, and added an end-to-end test protocol and full documentation.
- Designed and built the replacement backend, which transcribes speech, assembles sensor and conversation context, and streams synthesized speech back. Latency from end of speech to start of response dropped from about 2,000 ms to about 450 ms.

Web Developer – Project Serendib

Jan 2024 – Dec 2024

- Built a travel survey website and deployed it to AWS, revising the design with the client over several rounds of feedback.

Machine Learning Research Intern – Christopher Ré Lab, Stanford University

Jun 2023 – Sep 2023

- Added multimodal input support to a PyTorch classification project, and prototyped LLM desktop tools for Calendar and Search.